ASSIGNMENT-20.2

2303A51064

B-16

TASK 1 – Password Security Enhancement

UNSECURE CODE:-

```
users = {}

def register():
    u = input("Username: ")
    p = input("Password: ")
    users[u] = p
    print("Registered")

def login():
    u = input("Username: ")
    p = input("Password: ")
    if users.get(u) == p:
        print("Login successful")
    else:
        print("Invalid credentials")

register()
login()
```

```
Username: LAHARI
Password: LAHARI123
Registered
Username: LAHARI
Password: LAHARI123
Login successful
```

SECURE CODE:-

```
users = {}

def hash_password(p):
    return hashlib.sha256(p.encode()).hexdigest()

def register():
    u = input("Username: ")
    p = input("Password: ")
    users[u] = hash_password(p)
    print("Registered")

def login():
    u = input("Username: ")
    p = input("Password: ")
    if users.get(u) == hash_password(p):
        print("Login successful")
    else:
        print("Invalid credentials")

register()
login()
```

```
Username: ROHAN
Password: ROHAN22
Registered
Username: ROHAN
Password: ROHAN22
Login successful
```

TASK 2 – Secure API Key Management

UNSECURE CODE:-

```python
import requests

API_KEY = "12345"

url = f"https://api.example.com/data?key={API_KEY}"
print("Fetching data...")
```

```
Fetching data...
```

SECURE CODE:-

```python
import os
import requests

API_KEY = os.getenv("API_KEY")

url = f"https://api.example.com/data?key={API_KEY}"
print("Fetching data securely...")
```

```
Fetching data securely...
```

TASK 3 – Weak Encryption Detection

WEAK CODE:-

```python
import base64

text = "hello"
enc = base64.b64encode(text.encode())
print(enc)

dec = base64.b64decode(enc).decode()
print(dec)
```

```
b'aGVsbG8='
hello
```

SECURE CODE(FERNET):-

```python
from cryptography.fernet import Fernet

key = Fernet.generate_key()
f = Fernet(key)

text = "hello".encode()

enc = f.encrypt(text)
print(enc)

dec = f.decrypt(enc)
print(dec.decode())
```

```
b'gAAAAABp1MKMKK4_vydCt5kpE_j3amc_dJ1vAAQIYPTyYKifaQxNw
hello
```

TASK 4 – Security Review (Web Service)

INSECURE CODE:-

```python
from flask import Flask, request

app = Flask(__name__)

@app.route("/login", methods=["POST"])
def login():
    username = request.form["username"]
    password = request.form["password"]

    if username == "admin" and password == "123":
        return "Success"
    return "Fail"

app.run()
```

```
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
```

SECURE CODE:-

```python
from flask import Flask, request
import hashlib

app = Flask(__name__)

users = {"admin": hashlib.sha256("123".encode()).hex

@app.route("/login", methods=["POST"])
def login():
    username = request.form.get("username")
    password = request.form.get("password")

    if not username or not password:
        return "Invalid input"

    hashed = hashlib.sha256(password.encode()).hexdi

    if users.get(username) == hashed:
        return "Success"
    return "Fail"

app.run()
```

```
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
```

TASK 5 – Exception Handling

INSECURE CODE:-

```python
a = int(input("Enter number: "))
b = int(input("Enter number: "))

print(a / b)
```

```
Enter number: 5
Enter number: 6
0.8333333333333334
```

SECURE CODE:-

```python
try:
    a = int(input("Enter number: "))
    b = int(input("Enter number: "))

    result = a / b
    print("Result:", result)

except ValueError:
    print("Invalid input")

except ZeroDivisionError:
    print("Cannot divide by zero")

except Exception as e:
    print("Error:", e)
```

```
Enter number: 4
Enter number: 5
Result: 0.8
```